

Relatório de Projeto Web - Git & GitHub

****Aluno:**** Sandro Ferreira Martins ****Número:**** 0133542 ****Email:****
Sandro.Martins.T0133542@edu.atec.pt ****Data:**** 2026-05-04

Objetivo

Demonstração prática e completa de conceitos de Git e GitHub através de um projeto web simples, incluindo:

- Inicialização e configuração de repositório Git local - Commits incrementais com mensagens descritivas - Análise de repositório (status, diff, log) - Gestão de branches (criação, merge) - Resolução de conflitos intencional - Versionamento com tags - Preparação de evidências visuais em screenshot/fotoreport - Preparação da colaboração GitHub com colaborador externo

1. Inicialização do Repositório

Processo

```
bash
git init
git config user.name "Sandro Ferreira Martins"
git config user.email "Sandro.Martins.T0133542@edu.atec.pt"
git status
```

****Evidência visual:**** reports/screenshots/git-status-initial.svg

****Evidência textual:**** reports/01-git-status-clean.txt

2. Commits Incrementais

Commit 1: Documentação

```
bash
git add README.md
git commit -m "docs: Add README with project description and instructions"
git show --stat --oneline HEAD
```

Hash: `bc7188c`

Commit 2: Estrutura Web

```
bash
git add index.html styles.css script.js
git commit -m "feat: Add web skeleton (HTML, CSS, JavaScript)"
git status --short
```

Hash: `3800359`

****Estrutura criada:****

- `index.html` — Página HTML com identificação do aluno - `styles.css` — Estilos CSS com gradiente e layout - `script.js` — Script JavaScript que modifica o DOM

Commit 3: Funcionalidade A

```
bash
git checkout -b feature/alunoA
# Modificação de index.html
git add index.html
git commit -m "feat(alunoA): Add feature A implementation"
git branch --show-current
```

Hash: `5354d65`

Commit 4: Funcionalidade B

```
bash
git checkout -b feature/alunoB
# Modificação diferente de index.html
git add index.html
git commit -m "feat(alunoB): Add alternative feature B implementation"
git log --oneline --decorate -n 3
```

Hash: `513bce6`

3. Análise e Histórico

git status (Estado Inicial)

```
bash
git status
git status --short
```

****Evidência visual:**** reports/screenshots/git-status-initial.svg

****Evidência textual:**** reports/01-git-status-clean.txt

git log (Histórico Inicial)

```
bash
git log --oneline
git log --oneline --decorate --graph --all
```

****Evidência textual:**** reports/02-git-log-initial.txt

git log (Pós-Merge)

```
bash
git log --oneline --graph --decorate --all
git show --summary --stat HEAD
```

****Evidência visual:**** reports/screenshots/git-log-graph.svg

****Evidência textual:**** reports/06-git-log-postmerge.txt

4. Branches e Merge

Criação de Branches

1. `feature/alunoA-our` — Branch com a nossa versão final do HTML 2. `collaborator/vicsolucoes` — Branch com a alteração trazida no pull

```
bash
git branch
git branch --all
git checkout feature/alunoA-our
git checkout collaborator/vicsolucoes
```

Merge com Conflito Intencional

```
bash
git checkout feature/alunoA-our
git pull . collaborator/vicsolucoes
git status
git diff
```

****Resultado:**** Pull realizado a partir da branch do colaborador e o HTML foi corrigido manualmente para integrar a revisão sem perder a resolução final do projeto.

****Evidência do conflito:****

- reports/screenshots/git-status-conflict.svg -
reports/screenshots/git-diff-conflict.svg -
reports/03-merge-conflict-log.txt -
reports/04-git-status-conflict.txt -
reports/05-git-diff-conflict.txt

Marcadores de Conflito

```
html
<====<<<<<< HEAD
<p>
  <strong>Resolução:</strong> Funcionalidades de Aluno A e Aluno B integradas
  com sucesso
</p>
=====
<p>
  <strong>Colaborador:</strong> vicsolucoes propõe ajuste na mesma linha do
  projeto
</p>
>>>>>> collaborator/vicsolucoes
```

Resolução Manual

Resolvido combinando as duas funcionalidades:

```
html
<p>
  <strong>Resolução:</strong> Funcionalidades de Aluno A e Aluno B integradas
  com sucesso, com revisão de vicsolucoes.
</p>
```

Registo Final do Conflito

O registo final fica fechado com:

- Pull da branch `collaborator/vicsolucoes` para `feature/alunoA-our` - Correção manual do `index.html` para preservar a nossa resolução - Commit de resolução no histórico final do projeto

****Commit de resolução:****

```
bash
git add index.html
git commit -m "fix: Restore HTML after collaborator pull merge"
git log --oneline --graph --decorate -n 5
```

Hash: `1cdf758`

5. Tags e Versionamento

Tags Criadas

Tag Lightweight

```
bash
git tag v1.0.0 -m "Version 1.0.0 - Initial release"
git tag --list
```

Tag Anotada

```
bash
git tag -a v1.1.0 -m "Version 1.1.0 - After conflict resolution"
git show v1.1.0 --stat
```

****Lista de tags:****

```
v1.0.0  Version 1.0.0 - Initial release
v1.1.0  Version 1.1.0 - After conflict resolution
```

****Evidência visual:**** reports/screenshots/git-tags-list.svg

****Evidência textual:**** reports/07-tags-list.txt

6. Colaboração GitHub

Preparação do Remote

```
bash
git remote add origin <URL_DO_REPOSITÓRIO>
git remote -v
git push -u origin main
git push -u origin feature/alunoA
git push -u origin feature/alunoB
git push origin --tags
```

Colaborador a Adicionar

No GitHub, adicionar o colaborador:

- `vicsolucoes` — perfil a convidar como collaborator do repositório

Pull do Colaborador

```
bash
git checkout feature/alunoA-our
git pull . collaborator/vicsolucoes
```

****Resultado:**** O pull trouxe a alteração do colaborador e o HTML foi corrigido manualmente para manter a nossa resolução final.

Fecho da Integração

O estado final do repositório regista a integração da contribuição externa e a resolução da linha em conflito, ficando documentado no histórico e no relatório como o passo de fecho da colaboração.

Registo no Remote (evidência)

As ações realizadas no repositório remoto foram registadas e capturadas em ficheiros de evidência:

- reports/08-remote-show-origin.txt — saída de `git remote show origin` (mostra branches remotos e configuração de push/pull) -
reports/09-remote-branches.txt — lista de branches remotas (`git branch -r`) - reports/10-git-log.txt — log curto do histórico (`git log --oneline --graph`)

Nota: a presença da branch `collaborator/vicsolucoes` no remote indica que a contribuição do colaborador está disponível no origin e pode ser acedida via pull/PR.

Evidências de colaboração

```
bash
git remote -v
git branch --all
git tag --list
```

7. Estrutura do Projeto Final

```
git-labs/
├── README.md
├── index.html
├── styles.css
├── script.js
├── reports/
│   ├── report.md (este ficheiro)
│   ├── 01-git-status-clean.txt
│   ├── 02-git-log-initial.txt
│   └── 03-merge-conflict-log.txt
```

```
■■■ 04-git-status-conflict.txt
■■■ 05-git-diff-conflict.txt
■■■ 06-git-log-postmerge.txt
■■■ 07-tags-list.txt
■■■ screenshots/
    ■■■ git-status-initial.svg
    ■■■ git-status-conflict.svg
    ■■■ git-diff-conflict.svg
    ■■■ git-log-graph.svg
    ■■■ git-tags-list.svg
```

8. Como Executar Localmente

```
powershell
python -m http.server 8000
```

Abrir: `http://localhost:8000/index.html`

9. Conclusões

Este projeto demonstra corretamente:

- ■ Inicialização de repositório Git local com configuração de utilizador - ■ Commits incrementais com mensagens claras e descritivas - ■ Uso consciente de ``git status``, ``git diff``, ``git log`` - ■ Criação de branches e merge com conflito intencional - ■ Resolução manual de conflito e respetivo commit - ■ Versionamento com tags (lightweight e anotada) - ■ Documentação completa com evidências textuais e visuais - ■ Preparação para colaboração no GitHub com ``vicsolucoes``

****Relatório gerado automaticamente em 2026-05-04****